2조

3주차
조별활동 발표

2조 - 김기훈 황지윤 이용준 오유선





2조

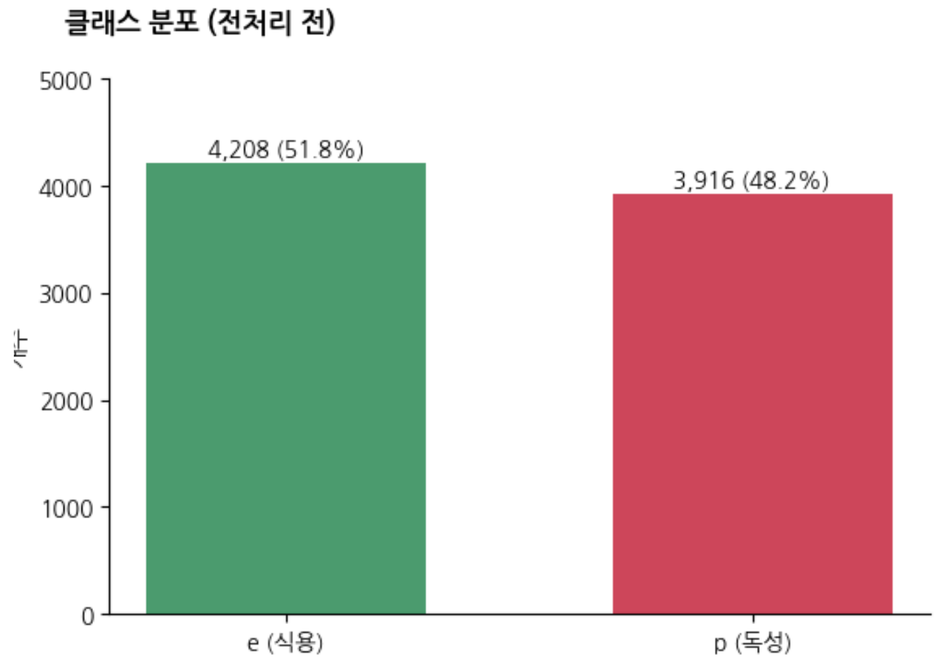
PART 1 · 분류 프로젝트

버섯 식용 / 독성 분류

분류 문제 — 버섯의 22가지 특징으로 '먹을 수 있는지(e)'와 '독성(p)'을 맞히기

2조 김기훈 · 황지윤 · 이용준 · 오유선

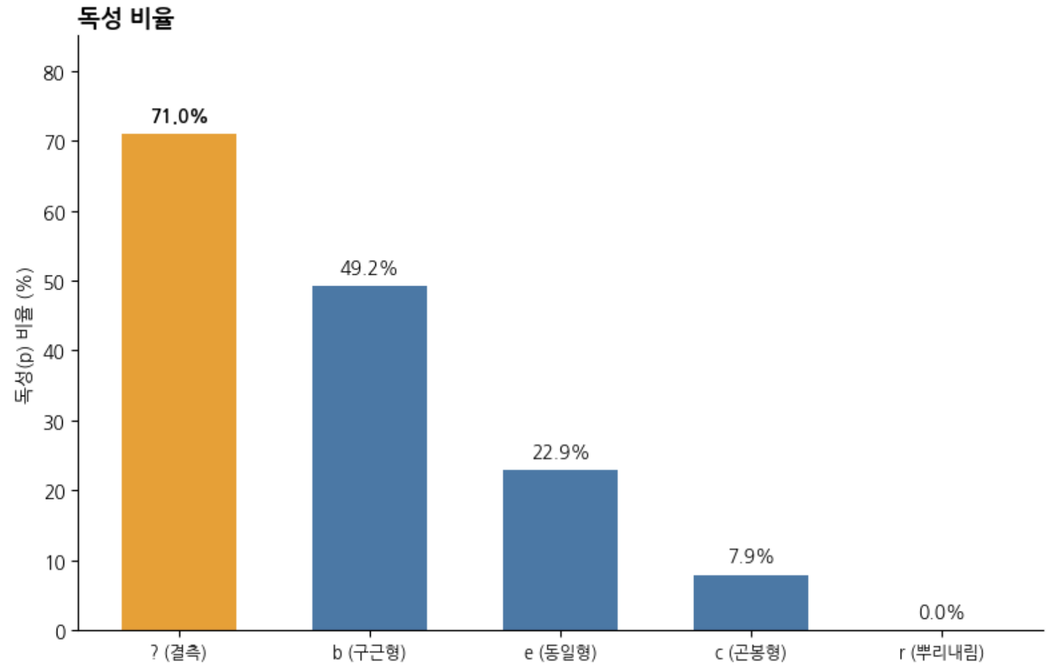
1. 전처리 전 · 클래스 분포



class	개수	비율
e (식용)	4,208	51.8%
p (독성)	3,916	48.2%

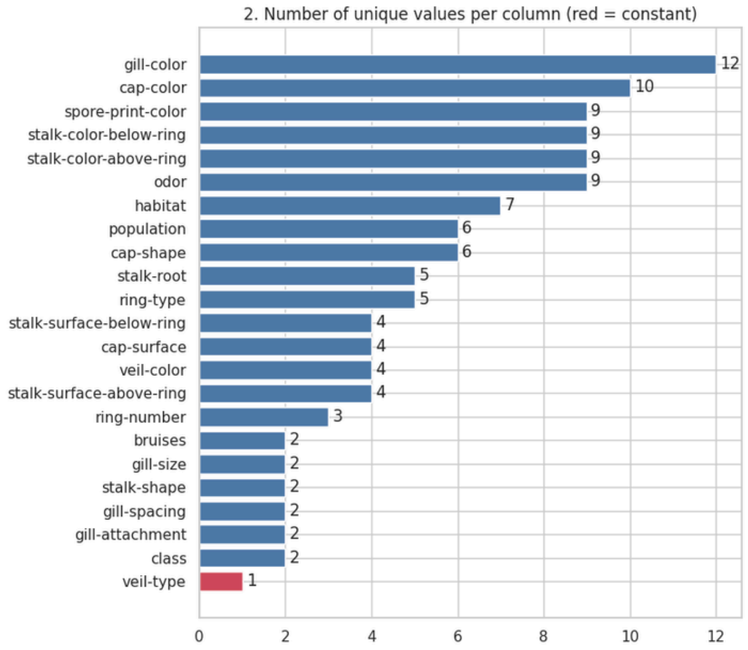
→ 식용과 독성이 거의 반반이라 불균형 처리 없이 진행, 모두 범주형이라 인코딩은 필수 (e : 1, p : 0)

2. 전처리 전 · stalk-root 독성 비율



→ 결측 자체가 독성 신호. ‘?’ 행을 지우면 독성 버섯의 약 45%가 함께 사라짐
'?'를 지우지 않고 하나의 범주로 유지

3. 전처리 전 · 칼럼별 고유값 개수



→ veil-type 값이 1개라 정보가 없어 삭제. 나머지는 범주 수가 작아 원-핫 인코딩 가능

4. 타깃 · 특성 인코딩

특성 21개 (X)

원-핫 인코딩

odor 값	a	c	f	l	m	n …
레이블	0	1	2	3	4	5 …

- 레이블 인코딩을 쓰면 'n(5)이 a(0)보다 크다'는 없는 크기 관계가 생김
- 원-핫 인코딩은 값마다 0/1 칼럼을 따로 만들어 이런 관계가 없음
- 로지스틱 회귀처럼 숫자 크기를 쓰는 모델과도 비교할 수 있게 원-핫 인코딩 선택

→ 21개 칼럼 → 116개 칼럼 (값 1개당 칼럼 1개)

```
pd.get_dummies(df.drop('class', axis=1), dtype=int)
```

정답 class (y)

레이블 인코딩

e → 0	p → 1	1개
식용 4,208개	독성 3,916개	정답 칼럼 수

- class는 모델이 맞혀야 할 정답이라 칼럼 하나(0/1)로 바꿈
- 원-핫 인코딩하면 class_e, class_p 두 칼럼이 생겨 정답으로 쓸 수 없음
- LabelEncoder는 알파벳 순서로 e=0, p=1을 부여

→ 정답은 0/1 숫자 하나로

```
df['class'] = encoder.fit_transform(df['class'])
```

5. 전처리 전후 Shape

전처리 전
(8124, 23)
원본 mushrooms.csv

→

전처리 후
(8124, 117)
특성 116개 + class 1개

삭제한 행
0건

단계별 shape (행, 열)

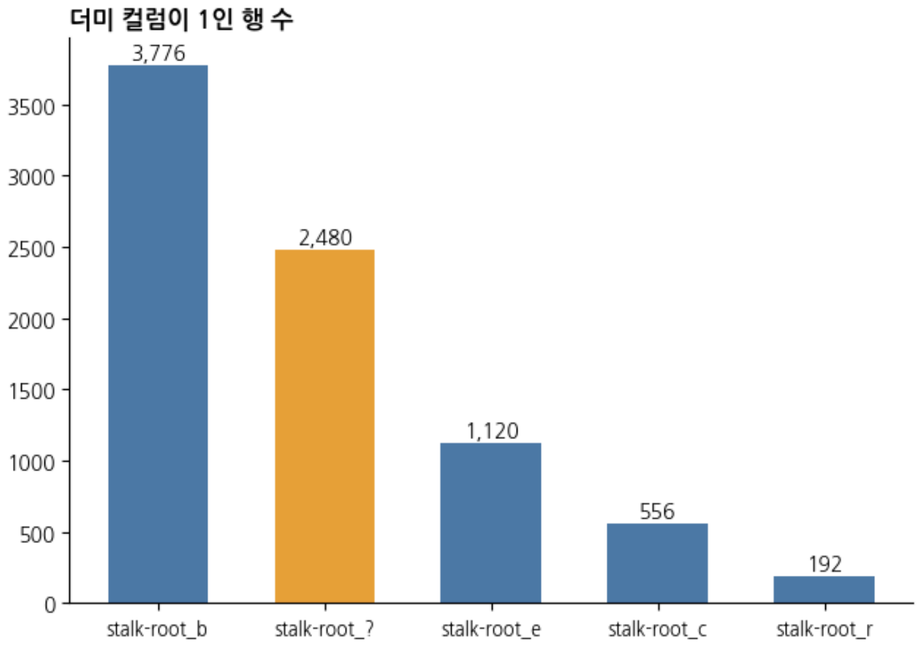
단계	shape	변화
원본	(8124, 23)	-
02 결측(?) 유지	(8124, 23)	값 그대로 (행 유지)
03 veil-type 삭제	(8124, 22)	− veil-type
04 class 레이블 인코딩	(8124, 22)	e→0, p→1
04 특성 원-핫 (X)	(8124, 116)	21열 → 116열
04 정답 분리 (y)	(8124,)	class 1열
X + y 저장	**(8124, 117)**	**preprocessed.csv**

변수별 값(범주) 개수 → 원-핫 칼럼 수

변수	고유값	원-핫 후
gill-color	12	12열
cap-color	10	10열
odor	9	9열
stalk-root	5	5열 (? 포함)
bruises	2	2열
veil-type	1	삭제
※ 21개 칼럼의 고유값을 모두 더하면 116 = 원-핫 후 칼럼 수		

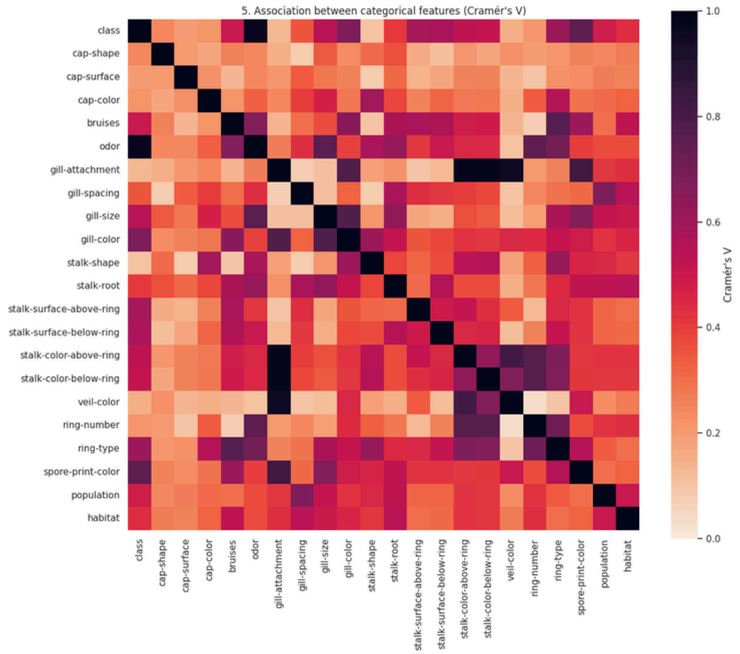
→ 행은 하나도 지우지 않음 | 열 23 → 22 (veil-type 삭제) → 특성 116 + class 1 = 117

6. 전처리 후 · stalk-root 더미 컬럼



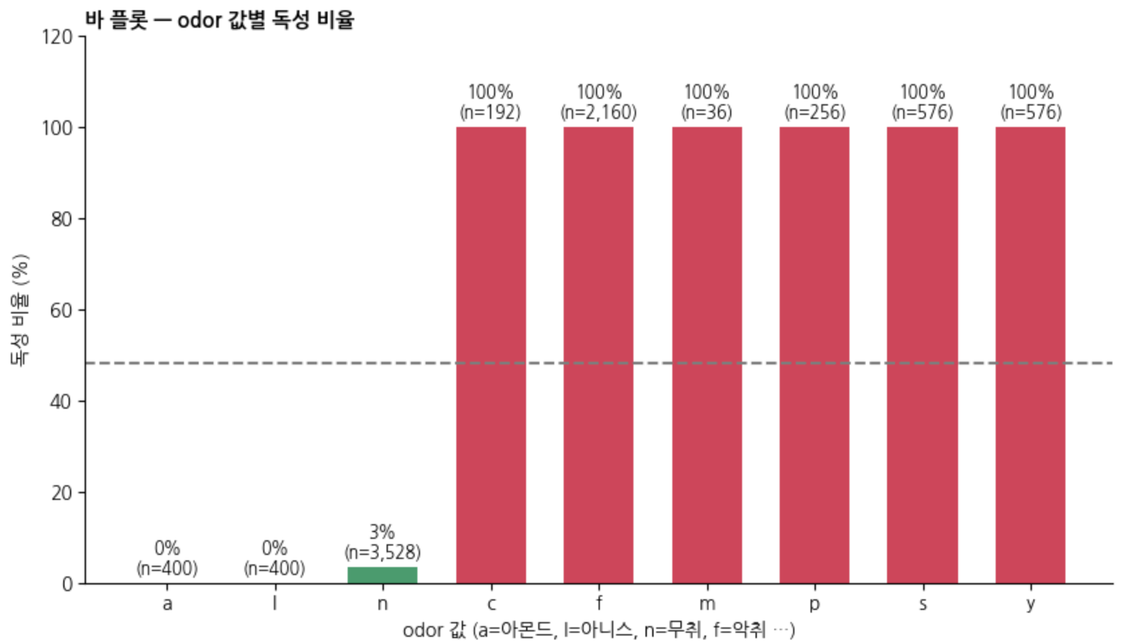
→ 원-핫 인코딩 이후에도 stalk-root_? 컬럼에 2,480행이 남아 있음
? 가 하나의 범주로 보존되어 결측 정보가 유지됨

7. 전처리 후 · 변수 간 연관성



→ 변수와 class의 연관도/ odor가 0.97로 판별을 주도함

8. 전처리 후 · odor 값별 독성 비율



→ 9개 값 중 8개는 독성 100% 또는 0%/ n(무취) 3,528개 중 3.4%가 독성
냄새만으로 대부분 판별되고, 냄새 없음(n) 그룹의 독성 120개는 포자 색으로 1차 구분 → 녹색(r)은 전부 독성, 흰색(w)만 추가 특성 필요

9. 다음 계획

사용할 모델 후보 (2주차 대표 분류 알고리즘)

나이브 베이즈	SVM	KNN	로지스틱 회귀	랜덤 포레스트	결정 트리
확률 기반	경계 기반	거리 기반	선형 · 비교용	앙상블 · 주요 후보	주요 후보

오른쪽일수록 우선 후보 → 범주형·원-핫 데이터에 강한 결정 트리 계열부터 시작하고 나머지와 비교

1단계

학습·테스트 분할

클래스 비율을 유지하며 학습 8 : 테스트 2로 분할

→

2단계

교차 검증 · 튜닝

여러 모델과 하이퍼 파라미터를 학습 데이터 안에서 비교

→

3단계

테스트 세트 평가

정확도 + 독성 재현율, 중요 특성(odor 등) 확인

`train_test_split → GridSearchCV`

2조

PART 2 · 회귀 프로젝트

자동자보험 사고 빈도 예측

분류에서 회귀로 - 이번엔 ‘무엇인지’ 가 아닌 ‘몇 건인지’를 맞힙니다.

2조 김기훈 · 황지윤 · 이용준 · 오유선

데이터 개요

freMTPL2freq.csv 프랑스 자동차 책임보험 데이터 · 한 행 = **자동차보험 계약 1건**

678,013	12	0
행 (계약 수) IDpol 모두 고유	**열** 수치형 8 · 범주형 4	**결측치 · 중복 행** 행 삭제 없이 전처리 가능

열 구성 — 역할별

타깃	**ClaimNb** 계약 기간 중 사고 건수 (0~16)
offset	**Exposure** 가입 기간 (0.0027~2.01년)
ID	**IDpol** 계약 번호 (확인용)

설명변수 9개

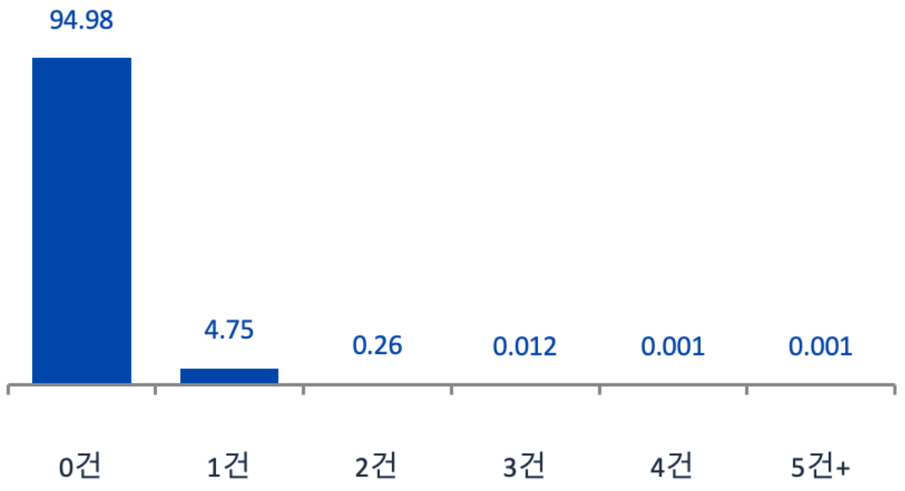
차량	VehPower · VehAge · VehBrand · VehGas
운전자	DrivAge · BonusMalus (할인·할증 등급)
지역	Area · Density · Region

→ 한쪽으로 치우친 변수가 많음 → 전처리에서 상한 · log 변환 · 범주화로 정리

BonusMalus 75%가 64 이하(최대 230) · Density 평균 1,792 vs 중앙값 393 · VehAge 최대 100년

타깃 ClaimNb · 평가 지표

사고 건수별 계약 비율 (%)



94.98%
사고 0건

0.0532건
계약당 평균

0.1007건
1년당 사고율

평가 지표

주 지표 **평균 포아송 deviance** — 작을수록 ↓

계약별 예측 건수가 실제와 얼마나 먼가

보조 **총 건수 비율** — 1에 가깝게

예측 합 ÷ 실제 합 — 전체 수준이 맞나

보조 **lift** — 클수록 ↑

예측 상위 20% ÷ 하위 20%의 실제 사고율

→ 개수 자료 → 회귀(포아송) 문제 · 정확도는 쓰지 않음 (모두 0건으로 예측해도 94.98%)

$$D = \frac{2}{n} \cdot \Sigma\left[\, y \cdot \log\left(\frac{y}{\hat{\mu}}\right) - (y - \hat{\mu}) \,\right], \quad \hat{\mu} = \text{Exposure} \times \text{예측 사고율}$$

01. 1년당 사고율 기준

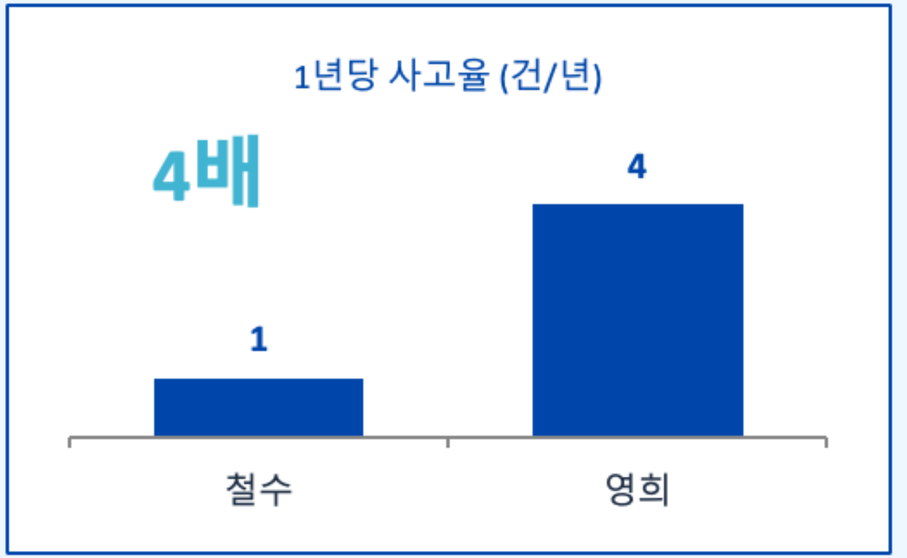
ClaimNb = Exposure × 1년당 사고율

사고 건수는 가입 기간이 길수록 늘어남
→ 위험은 '1년당 사고율'로 비교해야 함

	가입 기간	사고 건수	1년당 사고율
철수	1년	1건	1건 ÷ 1 = 1
영희	3개월 (0.25년)	1건	**1건 ÷ 0.25 = 4**

1 건수로 보면
철수 1건 = 영희 1건 → 똑같아 보임

2 1년당으로 보면
철수 1건 vs 영희 4건 → 4배



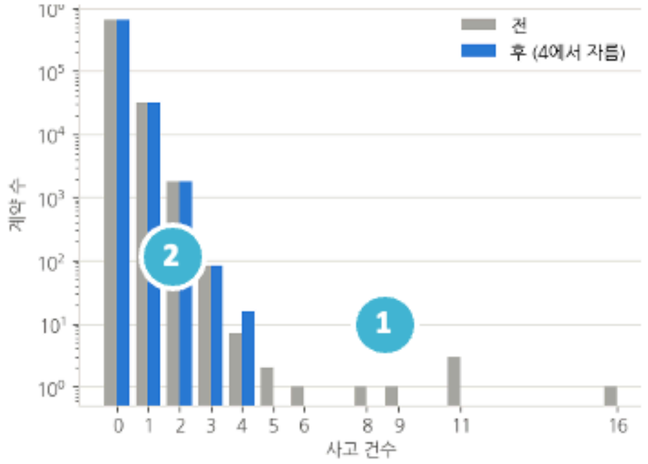
→ 사고 건수를 그대로 비교하지 않고, 가입 기간을 고려한 '1년당 사고율'을 위험의 기준으로 삼음

02. 사고건수·가입기간 상한

사고건수 (ClaimNb)

Target · 이상치 상한

전·후



1 5건 이상 9건
최대 16건. 9건 중 7건은 특성이 똑같아 중복 기록으로 추정

2 삭제 대신 4로
지우면 "위험한 계약" 신호까지 사라져 사고율 과소 추정

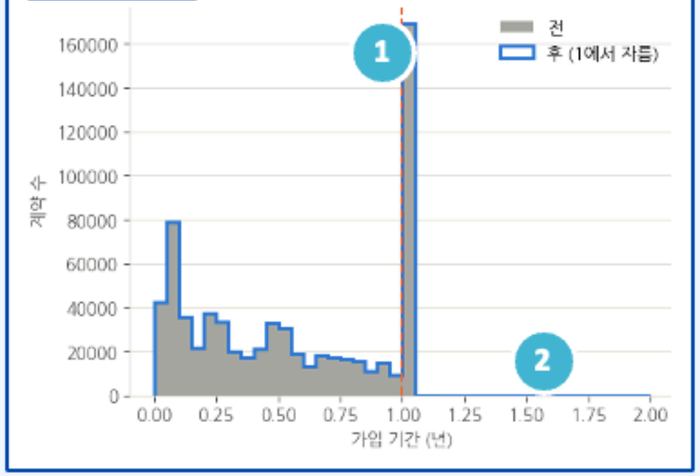
→ 삭제하지 않고 사고건수 4에서 상한

```
freq["ClaimNb"].clip(upper=4)
```

가입기간 (Exposure)

이상치 상한

전·후



1 1년 초과 계약
1,224건. 67%가 1.01~1.10년 → 날짜 기록 오차로 판단

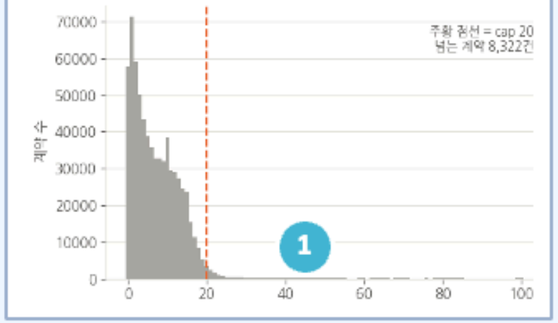
2 분모가 커지면
1건 ÷ 2.01년 = 0.50 → 실제의 절반으로 계산됨

→ 1년 단위 관측에 맞춰 1에서 상한

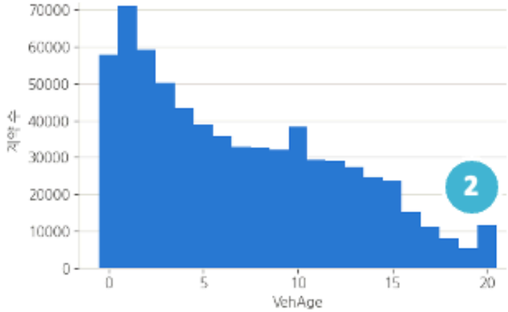
```
freq["Exposure"].clip(upper=1)
```

03. 차 나이·BonusMalus 상한

전



후

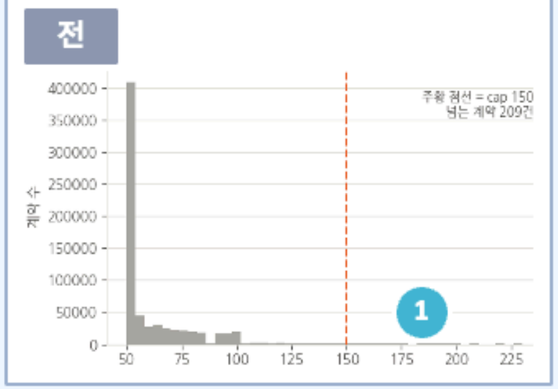


차 나이 (VehAge) 20에서 상한

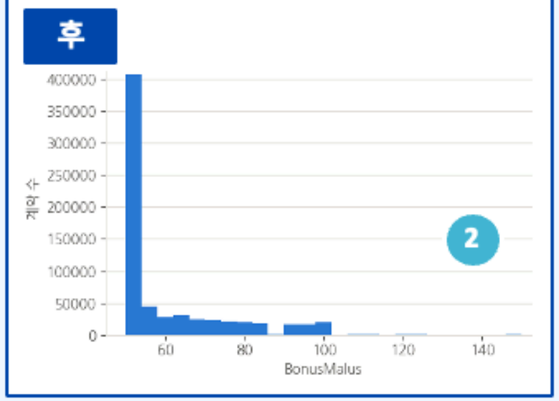
1. **21~100년에 1.2%만 흩어짐**
   나이 하나당 데이터가 적어 사고율이 0.024↔0.119로 들쭉날쭉
2. **20년 이상은 모두 20으로**
   행은 그대로 두고 값만 묶음 (8,322건)

```
freq["VehAge"].clip(upper=20)
```

전



후

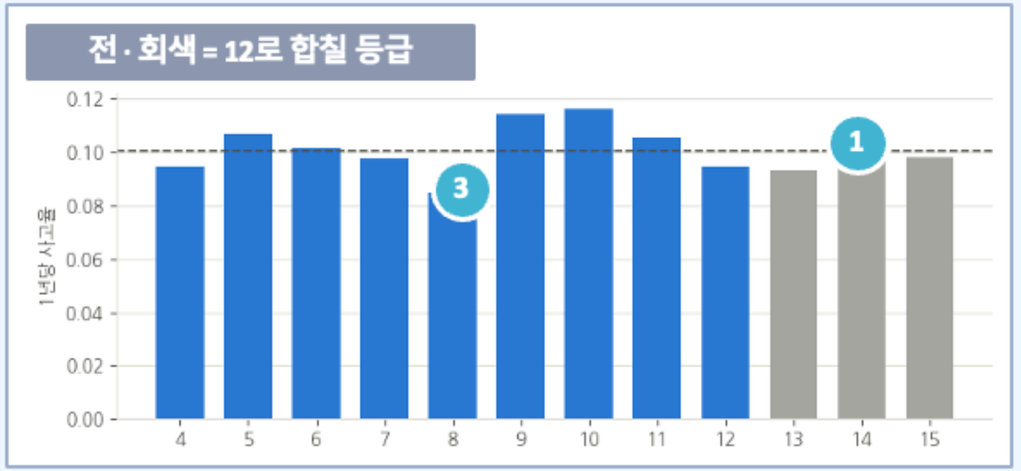


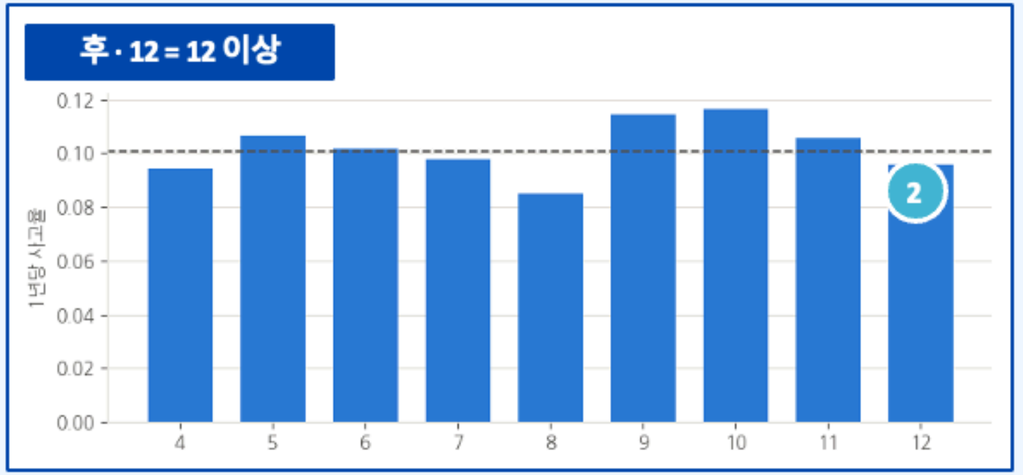
BonusMalus 150에서 상한

1. **150 초과 209건 (0.03%)**
   최댓값 230 → 계수가 끝의 극단값에 끌려감
2. **150에서 상한, 행은 유지**
   사고 위험을 가장 강하게 설명하는 변수라 '가장 위험한 그룹' 정보 보존

```
freq["BonusMalus"].clip(upper=150)
```

04. 차 힘 등급 범주화





1 13·14·15등급은 사고 수가 적음
각각 사고 119~153건뿐 → 사고율이 불안정

2 12등급과 사고율이 비슷
13~15등급 0.093~0.101 ≈ 12등급 0.095
→ 합쳐도 정보 손실 거의 없음

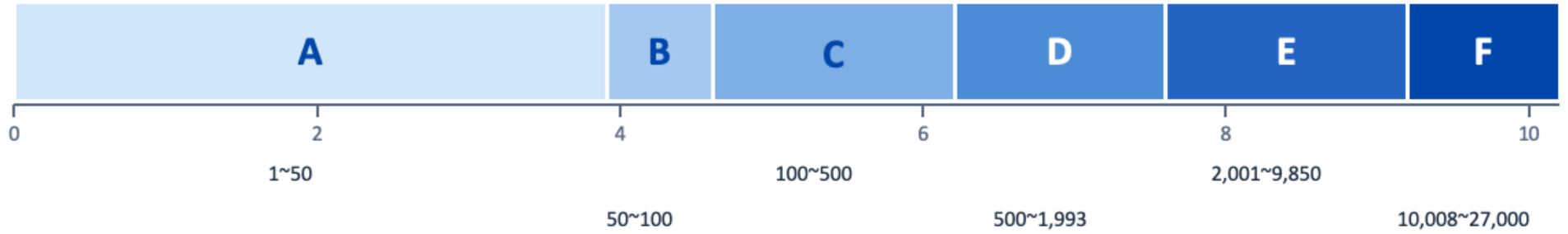
3 직선 관계가 아님
0.085 → 0.114 → 0.095처럼 오르내림 → 숫자 대신 범주로

→ 13 이상은 12로 합치고, 숫자가 아닌 범주(문자)로 넣음

```
freq["VehPower"] = freq["VehPower"].clip(upper=12).astype(str)
```

05. Area 열 삭제

Area별 Density 범위 — 가로축 log(인구 밀도) 위에 놓으면 6칸이 겹치지 않고 딱 나뉨



그대로 두면
같은 정보가 두 번 (다중공선성)
인구 밀도라는 같은 정보가 Area와 Density 두 변수에 중복

→

그 결과
SE 증가 · 해석 어려움
효과가 두 변수에 나뉘어 계수의 표준오차(SE)가 커짐

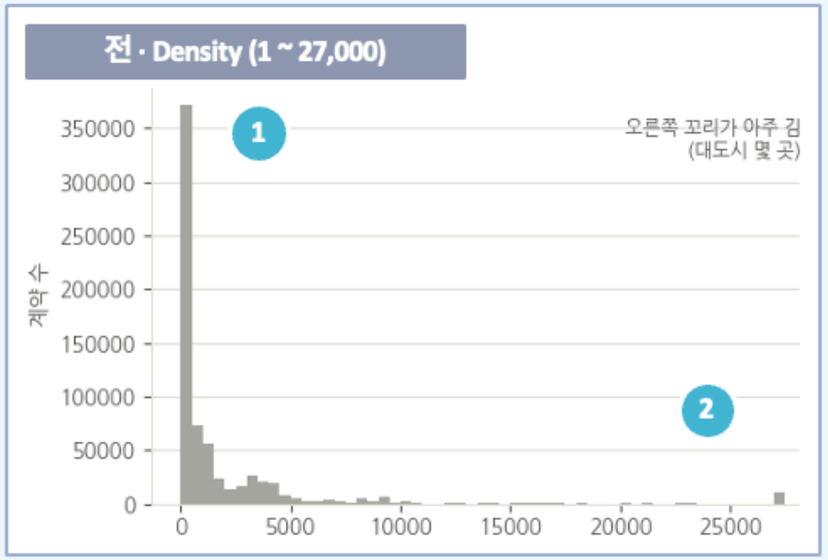
→

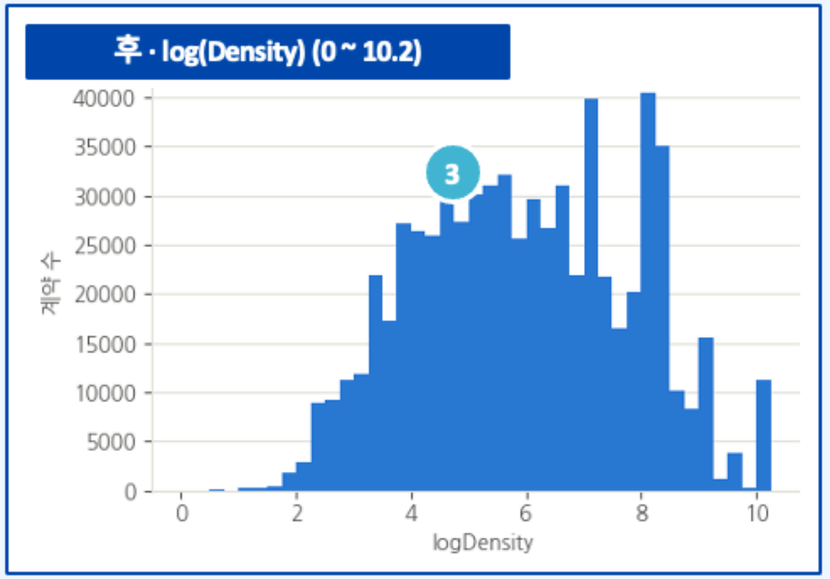
그래서
Density만 남김
더 자세한 정보인 Density만 남기고 Area는 삭제

→ Area는 Density를 6구간으로 자른 것일 뿐 → Area 열 삭제

```
freq.drop(columns=["Area"])
```

06. Density 로그 변환





1 낮은 값에 대부분 몰림
시골·소도시 계약이 왼쪽 끝에 집중

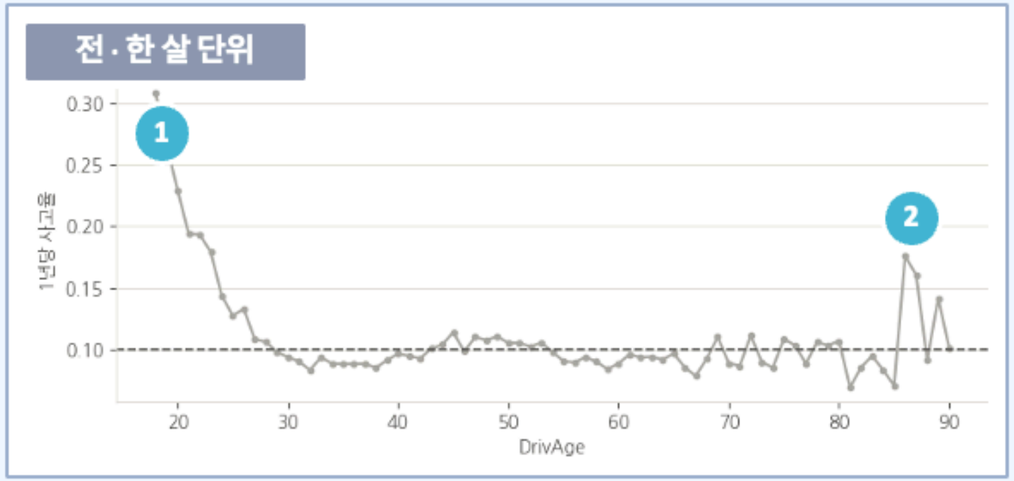
2 대도시 쪽 꼬리가 아주 김
27,000 같은 극단값이 계수를 좌우할 수 있음

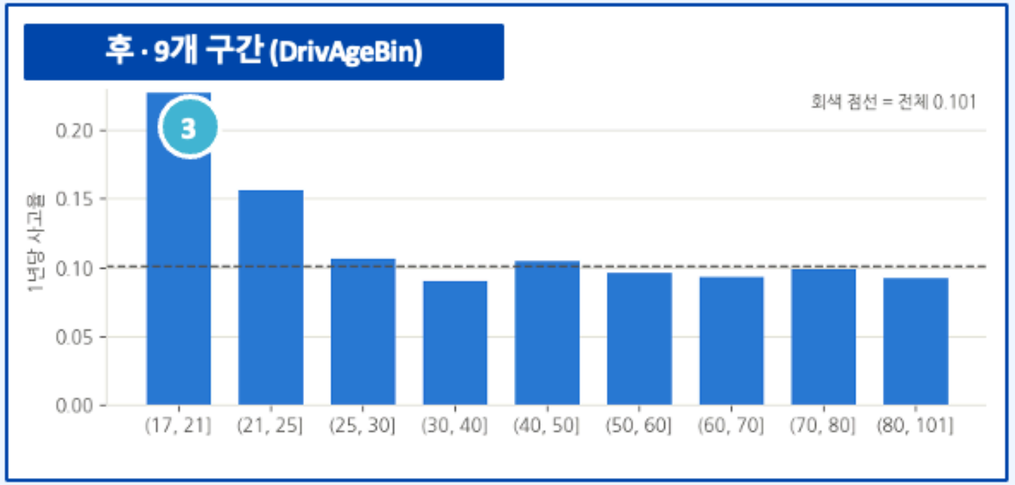
3 log를 씌우면 고르게 퍼짐
밀도 2배마다 사고율이 일정하게 오르는 곱셈 관계 → 직선 관계로

→ Density 대신 log(Density)를 모델에 넣음

```
freq["logDensity"] = np.log(freq["Density"])
```

07. 운전자 나이 범주화





1 직선 관계가 아님
18세 0.308 → 21세 0.194 → 30대 약 0.09로 급감

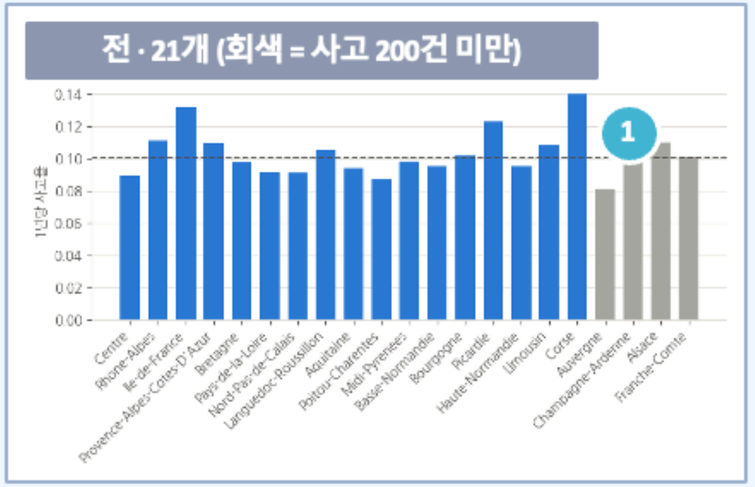
2 고령은 크게 흔들림
86세 0.176, 92세 0.000 — 나이별 가입 기간이 수십~수백 년뿐

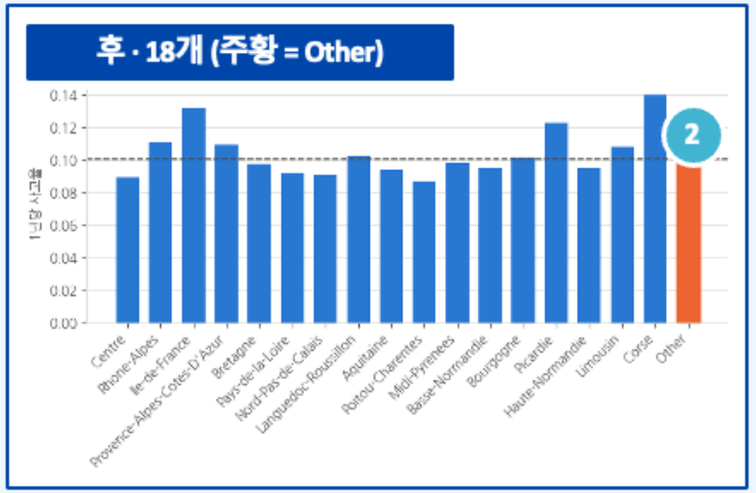
3 구간으로 묶으면 또렷
18~21세 0.22로 평균(0.101)의 2배 이상

→ 운전자 나이를 9개 구간으로 나눠 범주로 넣음

```
AGE_BINS = [17, 21, 25, 30, 40, 50, 60, 70, 80, 101]   # pd.cut(DrivAge, AGE_BINS)
```

08. 희소 범주 Other 통합





사고 200건 미만 수준	사고 수	1년당 사고율
Franche-Comte	57	0.101
Alsace	133	0.110
Champagne-Ardenne	160	0.133
Auvergne	189	0.081
B14 (브랜드)	166	0.073

사고율의 흔들림 ≈ 1 ÷ √사고 수

±26%
Franche-Comte (사고 57건)

±14%
사고 200건 기준

1 **행 수가 아닌 사고 수로**
95%가 0건이라 행이 많아도 사고가 적으면 못 믿음

2 **Other로 묶어 함께 추정**
지역 4개 + 브랜드 B14 → 하나의 계수

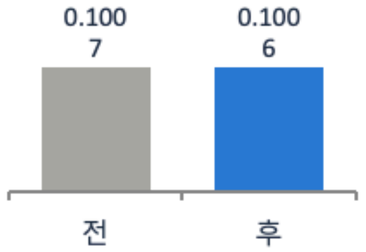
```
freq["Region"].where(freq["Region"].isin(keep_region), "Other")
```

09. 전처리 전후 Shape

전처리 전
(678,013, 12)
원본 freMTPL2freq

→

전처리 후
(678,013, 11)
모델에 넣을 열만 남김



삭제한 행 0건
1년당 사고율
0.1007 → 0.1006
거의 그대로 유지

단계별 shape (행, 열)

단계	shape	변화
원본	(678,013, 12)	-
02·03·04 상한	(678,013, 12)	값만 바꿈 (행 유지)
05 Area 삭제	(678,013, 11)	− Area
06 Density 로그	(678,013, 12)	+ logDensity
07 나이 범주화	(678,013, 13)	+ DrivAgeBin
08 Other 통합	(678,013, 13)	범주만 합침
모델용 열 선택	**(678,013, 11)**	**− Density, − DrivAge**

변수별 값(범주) 개수 변화

변수	전	후
VehPower	12	9 (범주)
VehAge	78	21
BonusMalus	115	93
DrivAge → Bin	83	9 구간
Region	21	18 (Other 포함)
VehBrand	11	11 (B14 → Other)

→ 행은 하나도 지우지 않고 열만 12 → 11로 정리 | 이후 8:2 분할: train (542,410, 11) / test (135,603, 11)

모델 후보군

단순한 모델 → 개수 자료 모델 → 약점 보완 모델 → 복잡한 모델 순으로 6개 후보를 비교

01 기준선

선형회귀 (OLS)

가장 단순한 비교 기준
→ 음수 예측 가능 · 개수 자료와 맞지 않음

02 기준선

로지스틱 회귀

95%가 0건 → 사고 유/무로 단순화
→ 건수(2건 이상)·가입 기간 정보 손실

03 기본 모델

Poisson GLM

개수 자료의 표준 · offset = log(Exposure)
→ exp(β) = 사고율 배수 → 해석 쉬움

04 보완

Quasi-Poisson · 음이항

분산 > 평균(과대산포)일 때 보정
→ 계수는 비슷, 표준오차를 현실적으로

05 보완

영과잉 포아송 (ZIP)

0건 94.98% — 0이 과잉인지 반영
→ Poisson보다 나은지 비교로 확인

06 상한선

트리 부스팅

XGBoost · LightGBM — 비선형·교호작용
→ 예측력 상한선 · 해석은 어려움

→ 해석 가능한 Poisson GLM을 기본 모델로, 나머지는 비교 · 보완용으로 사용

향후 진행 계획

사용할 모델 후보 (개수 자료 · 1년당 사고율 예측)

선형회귀 (OLS)	로지스틱 회귀	영과잉 포아송	트리 부스팅	음이항 GLM	Poisson GLM
기준선	유/무 단순화 · 비교용	0건 과잉 반영	예측력 상한선	과대산포 보완 · 주요 후보	기본 모델

오른쪽일수록 우선 후보 → 개수 자료와 offset에 맞는 Poisson GLM부터 시작하고 나머지와 비교

1단계

기본 모델 학습

train 542,410건으로 Poisson GLM 학습, 가입 기간은 offset = log(Exposure)로 넣음

→

2단계

과대산포 점검 · 비교

분산 > 평균이면 음이항으로 보완하고, ZIP · 부스팅과 같은 조건에서 비교

→

3단계

테스트 세트 평가

평균 포아송 deviance + 총 건수 비율 · lift, 주요 요인 exp(β) 확인

`PoissonRegressor` → `mean_poisson_deviance`

2조

감사합니다

2조 - 김기훈 황지윤 이용준 오유선